

Assignment 09 – PyTest Integration with HTML Reporting

1. Module Name

02_Unit_Test_Frameworks

2. Experiment Title

PyTest Integration with HTML Reporting

3. Problem Statement

Integrate Selenium Python automation with the PyTest framework. The automation should use fixtures for browser setup and teardown, execute test cases from the terminal, generate an HTML test report, and capture a screenshot automatically when a test fails.

4. Objective

The objective of this experiment is to understand PyTest integration with Selenium Python. The test suite should demonstrate fixture-based WebDriver management, PyTest assertions, terminal execution, HTML reporting, and failure screenshot handling.

5. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- PyTest
- pytest-html
- Google Chrome and ChromeDriver
- VS Code
- PyTest fixtures
- Assertions
- HTML test reporting
- Screenshot capture
- Terminal execution

6. Source Code / Implementation Steps

1. Install Selenium, PyTest and pytest-html using requirements.txt.
2. Create a driver fixture in conftest.py to start and close Chrome for each test.
3. Add a PyTest hook to store the test result and identify failed test calls.
4. Capture a screenshot automatically when a test fails.
5. Create test_login.py containing valid and invalid login tests.
6. Use Selenium locators to enter the credentials and click Login.
7. Use assertions to verify successful login and invalid-login validation.
8. Run the suite from the terminal using pytest.
9. Generate a self-contained HTML report using pytest-html.
10. Store execution screenshots in the Screenshots folder and the HTML report in the Reports folder.

7. Output / Labelled Screenshots

The following screenshots are captured during execution and stored in **Source_Code/Screenshots/**:

- Screenshot 1 – Successful Login Test

- Screenshot 2 – Invalid Login Validation
- Screenshot 3 – PyTest Terminal Execution
- Failure screenshots are automatically generated by the fixture when a test fails.

The HTML report is generated in **Reports/pytest_report.html** after running the test suite.

8. Result

The Selenium login tests were integrated successfully with PyTest. A WebDriver fixture manages browser setup and teardown, assertions validate the expected results, and pytest-html generates an HTML execution report. The framework also includes automatic screenshot capture for failed test cases.

9. Observation

PyTest fixtures reduce repeated browser setup and teardown code. The pytest-html plugin provides a structured HTML representation of test execution, while the failure screenshot mechanism provides useful evidence when a test does not pass. Keeping these responsibilities separate makes the test suite easier to maintain.

10. Conclusion

This experiment provided practical experience in integrating Selenium Python with PyTest and HTML reporting. I learned how to create fixtures, execute tests from the terminal, use assertions, generate an HTML report, and capture screenshots automatically for failed tests. The assignment was completed successfully.